Diagnosing the Invariance–Resolution Trade-off in JEPA World Models for Control

May 25, 2026

Abstract

A common motivation for Joint-Embedding Predictive Architectures (JEPAs) is that latent prediction may reduce pressure to preserve observation-level details that are irrelevant to predicting future target representations, and may therefore encourage more abstract representations. We test this robustness boundary for JEPA world-model control using **LeWorldModel (LeWM)** [21] across PushT, TwoRoom, Reacher, and Cube, sweeping eight levels of train-time pixel-noise augmentation under a unified 3-seed × 100-trajectory evaluation protocol. Without noise-aware training, LeWM is highly fragile to test-time visual corruptions: PushT success drops from 86.33% on unperturbed evaluation images to 4.67% under Gaussian pixel noise applied to both observation and goal images (standard deviation, std = 0.08), and TwoRoom drops from 94.00% to 50.00%. Noise training can close much of this gap, but on our sweep grid we do not observe a single noise level that is jointly optimal across tasks: visually redundant TwoRoom prefers heavy noise, while contact-heavy PushT has distinct unperturbed and robustness optima. We frame this dissociation as an *invariance–resolution trade-off*: stronger invariance pressure suppresses nuisance pixel variation but can also compress the action-relevant distinctions that closed-loop planning requires, and task structure determines the balance. A five-layer diagnostic protocol traces the LeWM mechanism to representation compression, loss of transition-key resolution, and reduced controllability. A full Planning with Latent Dynamics Models (PLDM) replication shows the same task-level fragility/recovery signature but a different diagnostic profile, most consistently accompanied by reduced multi-step predictor drift while rank and resolution are largely preserved. Finally, cross-checkpoint diagnostics have a precise scope: the strongest *fragility ratio* predicts residual checkpoint quality on PushT after controlling for `std_max`, but it does not predict the unperturbed-to-corrupted gap; this null replicates on PLDM and on the joint LeWM+PLDM sample. We focus on diagnosis rather than a new training algorithm: the contribution is an empirical study of JEPA + Cross-Entropy Method (CEM) visual-corruption failure, a reproducible diagnostic toolkit, and a delineation of what cross-checkpoint diagnostics can and cannot predict.

Keywords: world models; JEPA; visual robustness; representation diagnostics; invariance–resolution trade-off.

1 Introduction

1.1 The implicit invariance heuristic and where it breaks

Since Yann LeCun proposed the Joint-Embedding Predictive Architecture (JEPA) [19], the paradigm has been advanced as a direction for self-supervised learning. Unlike generative models (variational autoencoders, diffusion), JEPA does not reconstruct pixels; it predicts future *representations* in latent space. A common motivating intuition is that latent prediction may reduce pressure to preserve observation-level details that are irrelevant to predicting future target representations, and

may therefore encourage more abstract representations. This intuition is reflected in the framing of the JEPA paper sequence [2, 4] and related work, but it is not a formal robustness guarantee.

This distinction matters because visual robustness has already been studied in adjacent settings: data augmentation is central to pixel-based reinforcement learning [34, 33, 14], and recent JEPA variants test robustness on image classification [23], synthetic nuisance distractors [15], medical ultrasound [24], or slow-feature distractors in JEPA-style planning [31]. What remains open is a more basic question: whether the invariances a latent predictive world model learns are the invariances that closed-loop control actually requires. Unlike a linear probe or a recognition test, planning needs the latent to suppress nuisance visual variation *while preserving action-relevant resolution* — the fine state distinctions that change which action is best. We study this tension empirically in JEPA-style world-model control with CEM planning, using task success as the operational criterion and representation diagnostics to localise where it breaks.

This leaves a basic operational question open: **if the input image is degraded by sensor noise, lighting change, or camera jitter, does a JEPA world model with Cross-Entropy Method (CEM) planning still act reliably?**

The empirical answer is no in our protocol. On PushT (2D pushing), the LeWM checkpoint trained without input-noise augmentation achieves 86.33% on unperturbed evaluation images but falls to 4.67% under Gaussian pixel noise with standard deviation (std) = 0.08 — near-random. TwoRoom (2D navigation) drops from 94.00% to 50.00%. Latent prediction alone does not, in this regime, confer the visual-corruption robustness one might hope for.

1.2 The core tension: no jointly optimal noise level on the sweep grid

A natural remedy is input-side noise augmentation during training, a technique long-validated in supervised and contrastive learning [6, 7]. But a deeper question arises: **does there exist a single, universal noise level that is optimal across tasks?**

A systematic sweep across four tasks at eight levels of `std_max` $\in \{0.01, \ldots, 0.08\}$ answers no:

- **TwoRoom** (visually redundant navigation): unperturbed success rises monotonically with noise, peaking at `std_max` = 0.08 (98.33% / 98.67%).
- **PushT** (contact-heavy manipulation): unperturbed success peaks at `std_max` = 0.03 (89.67%), but robustness at px+goal 0.08 peaks at `std_max` = 0.06 (87.00%) — unperturbed and corrupted-evaluation optima dissociate.
- **Reacher** (continuous reaching): lies on a 0.02–0.06 plateau; the unperturbed point-best is at `std_max` = 0.06 (86.00%), while px+goal 0.08 point-best is at `std_max` = 0.02 (85.67%). Very low noise (0.01) is statistically indistinguishable from base.
- **Cube** (structured 3D manipulation): noise sweep is the weakest of the four, with no monotone trend on unperturbed evaluation. We read this as the trade-off's mildest manifestation rather than an absence: a structured action sequence and predictable visual-action coupling supply enough robustness that input-side augmentation buys comparatively little.

This exposes a fundamental tension: **global noise augmentation cannot distinguish "background visual redundancy that should be made invariant" from "control-relevant features that should retain resolution".**

1.3 Contributions

C1. A systematic quantification of visual-corruption fragility of JEPA + CEM control across four representative tasks. We sweep 4 tasks × 8 noise levels, and evaluate all 36 checkpoints under a unified 3-evaluation-seed × 100-trajectory protocol, for 300 trajectories per condition.

The same sweep replicates on a second method family, Planning with Latent Dynamics Models (PLDM), confirming that the per-task signatures of the failure mode — visually redundant TwoRoom moves into a high-noise plateau, contact-heavy PushT has a steep no-noise-training cliff with large noise-training recovery, structured 3D Cube responds weakly — are not LeWM-specific (Section F).

C2. The "invariance–resolution trade-off" concept and a five-layer diagnostic protocol that operationalises it. The protocol covers encoder shift, encoder geometry, predictor sensitivity, latent-noise response, and task resolution, and bundles 17 concrete metrics (Section 3.3). We validate it by Spearman correlations on the 4-task $\times$ $n = 9$ LeWM sweep under unified 3-seed $\times$ 100 evaluation, with partial correlations conditioned on training noise to separate residual checkpoint-quality signal from the monotone sweep trend induced by `std_max`.

C3. A mechanistic account of why global noise augmentation has limited returns, on the LeWM sweep. On TwoRoom (visually redundant) the gains coincide with desirable representation compression — effective rank drops by roughly 30% (47.60 $\rightarrow$ 33.59) and the more compact latent is easier to plan in. On PushT (contact-heavy), the base representation already has higher rank and higher controllability (effective rank 76.42; inverse-dynamics probe $R^2 \approx 0.77$); even the light representative noise-trained checkpoint compresses rank by about 44% (76.42 $\rightarrow$ 42.85) with concomitant downward movement in transition-resolution metrics and the controllability probe (both drop by a few points; see Table 4), so further compression cannot be assumed safe for contact-heavy tasks. The PLDM cross-method check in Section F reproduces the task-level recovery signature but with a different diagnostic profile, most consistently accompanied by reduced multi-step predictor drift while rank and resolution are largely preserved. This makes the LeWM compression chain a method-specific mechanism rather than a universal claim.

C4. An explicit delineation of cross-checkpoint diagnostics. The strongest cross-checkpoint diagnostic (the fragility ratio) carries a residual checkpoint-quality signal beyond the sweep-level `std_max` effect (partial Spearman $\rho = -0.59$ on unperturbed evaluation and -0.41 on px+goal 0.08 after conditioning on `std_max`, on the $n = 9$ LeWM PushT sweep with unified 3-seed $\times$ 100 eval; 95% bootstrap confidence intervals (CIs) $[-0.97, -0.10]$ and $[-0.77, +0.00]$ respectively). However, the apparent $\rho(\text{metric}, \text{corruption drop}) = -0.77$ is mediated by `std_max`: after partialling out `std_max`, that correlation collapses to $+0.06$ (95% CI $[-0.00, +0.25]$, including 0). The same absence of stable residual corruption-gap association appears on PLDM PushT ($\rho = -0.78$ unconditional, -0.14 partial on `std_max`, $n = 9$; CI $[-1.00, +0.87]$) and in the joint LeWM+PLDM $n = 18$ sample ($+0.11$ partial on `std_max` and method; CI $[-0.54, +0.71]$); the intervals are wide and include zero. The metric is useful for mechanism localisation and checkpoint selection within a fixed training protocol, but it cannot replace corruption evaluation when the question is robustness across training protocols (Sections 4.5, 5.3 and F).

1.4 Organisation

Section 2 reviews related work; Section 3 defines the LeWM background, noise protocol, and diagnostic framework; Section 4 reports the experimental findings; Section 5 discusses mechanism and implications; Section 6 concludes.

2 Related Work

2.1 JEPA and latent world models

JEPA [19] predicts in latent space rather than reconstructing pixels. I-JEPA [2] predicts target representations from a masked context; V-JEPA [4, 3] extends the framework to video understanding and video-driven world modelling; LeWM [21] is the first end-to-end stable JEPA world model, using **SIGReg** (Sketch Isotropic Gaussian Regularizer) — random projections plus Epps–Pulley empirical-characteristic-function matching [8] — to prevent representation collapse without batch normalisation. LeWM is our baseline. The original LeWM paper reports a Violation-of-Expectation (VoE) experiment showing the model is sensitive to physical perturbations (object teleportation) but not to visual perturbations (colour change). VoE measures prediction error (surprise), not control success rate, and colour change differs from pixel-level Gaussian noise. We therefore focus on success-rate robustness under controlled test-time pixel corruptions.

For a second method family we evaluate PLDM [26, 27] as implemented in `stable-worldmodel` [20] on the full sweep grid (Section F). The PLDM sweep covers the same four tasks and the same eight values of `std_max` as our LeWM sweep, under the same 3-seed $\times$ 100-trajectory evaluation protocol. We use PLDM to test which findings replicate across method families and which are LeWM-specific; the trade-off concept and the diagnostic toolkit themselves are introduced and analysed on LeWM, and the LeWM canonical tables in the main text are kept method-pure for clarity.

2.2 Robustness studies of JEPA

N-JEPA [23] introduces diffusion-noise augmentation into I-JEPA, improving downstream classification robustness. Variational JEPA (VJEPA) [15] tests a synthetic Noisy-TV environment and reports that JEPA-family models maintain signal-recovery $R^2 > 0.84$ even at the highest distractor scale. US-JEPA [24] studies ultrasound representations, where low signal-to-noise ratio and speckle patterns are central acquisition challenges. The contemporaneous Bisim-JEPA [31] attacks the related problem of slow-feature distractors in JEPA-based planning by augmenting the predictive objective with a bisimulation encoder that maps states with similar dynamics to nearby latents. Their intervention (bisimulation regularizer for control-relevant invariance) is complementary to ours: we instrument an unmodified LeWM under pixel noise and report what its latents *do and do not* retain, while they propose an inductive bias that targets a different corruption family (slow-feature distractors). These studies show that JEPA robustness is an active topic; our narrower contribution is to evaluate JEPA-style latent world models under test-time pixel corruptions in closed-loop CEM control, with task success rates and cross-checkpoint diagnostic correlations. VJEPA's positive Noisy-TV result is not contradicted by our data: it uses a synthetic signal-recovery evaluation, whereas our failure mode appears under closed-loop control success.

2.3 World models and input augmentation

Input-side image augmentation is an established route to visual robustness in pixel-based reinforcement learning (RL): DrQ [34] and DrQ-v2 [33] regularise a model-free critic with random-shift augmentation; SODA / DeepMind Control Generalization Benchmark (DMC-GB) [14] formalises a visual-generalisation benchmark on DeepMind Control with random colours and video backgrounds. Both lines establish that augmentation pressure during training is a strong baseline for closing visual-corruption gaps — but they study model-free policies with explicit reward signals, not JEPA-style latent-prediction world models with planning-time CEM control. In the model-based RL world-model literature, DreamerV3 [12] and TD-MPC2 [13] rely on learned visual encoders

and latent dynamics, which may provide some implicit tolerance to benign visual variation but do not by themselves settle sensor-noise robustness. ViGMO [22] tests Gaussian noise and blur on DeepMind Control tasks, finds "sensor noise is a fundamentally different distribution shift", and proposes a latent-consistency loss. ViGMO addresses model-based RL, not JEPA architectures; its conclusion is directionally aligned with ours, but our contribution adds mechanistic decomposition via the five-layer diagnostic, not just performance reporting.

2.4 The invariance–resolution tension

The idea that representation learning balances two competing properties is well established in the self-supervised literature. Wang and Isola [32] decompose the contrastive loss into an *alignment* term (positive pairs close together) and a *uniformity* term (features spread on the hypersphere), and show that downstream linear-probe accuracy tracks both in concert. Tamkin et al. [29] argue, in the contrastive-learning context, that label-destroying augmentations can be useful — augmentations act as feature dropout rather than pure invariance inducers. Zhang and Ma [35] note that augmentation modules can impose invariances that must be matched to the task. Concurrent seq-JEPA [11] resolves a related *invariance–equivariance* tension in JEPA by introducing architectural disentanglement of two heads. Our contribution is conceptually adjacent but operationally distinct: we study a *compression-versus-control-resolution* trade-off observed empirically in latent predictive world models, where the downstream task is planning rather than discrimination, and we operationalise it through a diagnostic protocol rather than a new architecture or loss.

2.5 Representation diagnostics and collapse analysis

Self-supervised learning broadly uses effective rank [25], condition number, and participation ratio to diagnose dimensional collapse [16]. Next-Latent Prediction [30] uses effective latent rank to assess world-model compactness. VICReg [5] provides an anti-collapse regularizer distinct from SIGReg. Centered Kernel Alignment (CKA) [17] compares representations across networks or training conditions; we use it to compare unperturbed-input and noisy-input latents within a checkpoint. Linear probes for representation analysis follow Alain and Bengio [1]. Most relevantly, RankMe [10] showed that a label-free effective-rank diagnostic correlates with downstream linear-probe transfer across hyperparameter sweeps in self-supervised learning. Our cross-checkpoint correlation analysis (Section 4.5) asks the analogous question for control: *does a label-free predictor-sensitivity diagnostic correlate with downstream task success after the sweep-level training trend is removed?* Individual metrics are not new; our contribution is to combine them into a coherent five-layer protocol with per-token noise sensitivity, cross-checkpoint correlation, and a partial-correlation validation scheme conditioning on training noise — and to delineate where that programme succeeds and where it does not.

3 Background and Diagnostic Framework

3.1 LeWorldModel baseline

LeWM [21] is an end-to-end JEPA world model trained with two terms only:

$$\mathcal{L}_{\text{LeWM}} \;=\; \mathcal{L}_{\text{pred}} \;+\; \lambda \cdot \mathcal{L}_{\text{SIGReg}}\,, \tag{1}$$

where $\mathcal{L}_{\text{pred}}$ is the latent-space mean-squared error (MSE) between predicted and target representations. SIGReg projects each latent onto M unit-norm random directions, computes the Epps–Pulley

empirical-characteristic-function distance [8] between each projection and $\mathcal{N}(0,1)$, and aggregates with Gauss-window weights, preventing collapse without explicit BatchNorm. (The Cramér–Wold theorem motivates the construction: equality of high-dimensional distributions reduces to equality of all one-dimensional projections of their characteristic functions.) Inference uses the Cross-Entropy Method (CEM) [18] for latent-space model predictive control (MPC) [9].

3.2 Input-side noise augmentation

We add per-frame Gaussian noise to the LeWM input pipeline via `utils.py::AddNormalizedGaussianNoise` (Section A). Each frame is independently noised: Bernoulli(p) decides whether the frame is corrupted, and if so the standard deviation is drawn from Uniform$(0, \text{std_max})$. We fix $p = 1.0$ and sweep `std_max` $\in \{0.01, \ldots, 0.08\}$ (eight levels). Evaluation uses two noised conditions: pixels+goal 0.05 and pixels+goal 0.08 (Gaussian noise of the indicated std applied to both pixels and the goal image).

3.3 Five-layer diagnostic framework

The framework decomposes the JEPA + CEM forward pass — pixels → encoder f → latent z → predictor g → cost surface → planned actions — into five sequential stages and instruments each transition with one or more metrics. Layers 1–2 characterise the encoder output; Layer 3 measures how the predictor amplifies an encoder shift; Layer 4 injects noise directly into the latent to isolate predictor and cost contributions; Layer 5 quantifies planning-relevant information retained in the latent. Most individual metrics already have a literature home (cited inline below). Our additions are scale-normalised ratios that compare perturbation effects with the unperturbed local nearest-neighbour scale. We use nearest-neighbour distance as a local latent scale primitive in the spirit of k-nearest-neighbour (kNN) out-of-distribution detection [28], but the ratios themselves are introduced here.

Operationalising the invariance–resolution trade-off. We use *invariance* and *resolution* as operational concepts measured by complementary subsets of the protocol. A latent representation is *invariant to nuisance visual variation* when perturbing pixels along a task-irrelevant axis leaves the encoded state within the same local latent neighbourhood; we approximate this through Layer 1 encoder-shift quantities (raw angle/L2 and the encoder-shift ratio r^{enc} normalised by the unperturbed nearest-neighbour scale) and through Layer 2 Centered Kernel Alignment (CKA) between unperturbed and corrupted latents. A representation *retains action-relevant resolution* when nearby but action-distinct states remain separable enough for the predictor and the planner; we approximate this through Layer 5 transition-resolution ratios R_d and the inverse-dynamics linear-probe R^2. Layer 3 (predictor sensitivity) and Layer 4 (latent-noise response) mediate between the two: they quantify how an encoder shift propagates through the predictor and how the cost surface reacts to direct latent perturbation. The trade-off arises because a single training-time pressure — input-side noise augmentation in our experiments — simultaneously reduces nuisance sensitivity (good for invariance) and can compress the distinctions the planner needs (bad for resolution). Table 9 (Section 5.2) summarises where each of our four tasks sits on this trade-off.

Minimal notation. Let $z_i = f(x_i)$ and $\tilde{z}_i = f(\tilde{x}_i)$ be unperturbed and noised latents for the same token. Let

$$d_{\cos}(a, b) = 1 - \frac{a^\top b}{\|a\| \, \|b\|}$$

and define the unperturbed nearest-neighbour (NN) scale

$$d_i^{\mathrm{NN}} = \min_{j \neq i} d_{\cos}(z_i, z_j).$$

The introduced ratios are

$$r_i^{\mathrm{enc}} = \frac{d_{\cos}(z_i, \tilde{z}_i)}{d_i^{\mathrm{NN}} + \epsilon},$$

$$r_i^{\mathrm{pred}} = \frac{d_{\cos}(g(z)_i, g(\tilde{z})_i)}{d_i^{\mathrm{NN}} + \epsilon}, \tag{2}$$

$$R_d = \frac{\mathrm{median}_t \; d(z_t, z_{t+1})}{\mathrm{median}_{(t,t') \in \mathcal{F}} \; d(z_t, z_{t'}) + \epsilon}. \tag{3}$$

Here r_i^{enc} is the encoder-shift ratio, r_i^{pred} is the fragility ratio, and R_d is the transition-resolution ratio for either cosine or L2 distance d. $\mathcal{F}$ denotes random far pairs from different temporal locations, and $\epsilon = 10^{-8}$ is a numerical-stability constant. Released JSON artifacts keep the implementation keys (`noise_to_nn_cos_ratio`, `predictor_target_to_nn_cos_ratio_at_max_std`, and `transition_resolution_ratio_{cos,l2}`), but the main text uses the shorter prose names above.

Layer 1 — Encoder shift. The magnitude and direction of latent displacement under input noise. Metrics: raw angular/L2 shift, angular gain per unit noise, and the encoder-shift ratio in Equation (2).

Layer 2 — Encoder geometry. The global structure of the encoded latent space. Metrics: local neighbourhood scale, effective rank [25], and Centered Kernel Alignment (CKA) [17] between unperturbed-input and noisy-input latents at the diagnostic's maximum injection std.

Layer 3 — Predictor sensitivity. How the predictor amplifies the encoder shift. Metrics: the *fragility ratio* in Equation (2) and multi-step rollout L2 drift at horizon T.

Layer 4 — Latent-noise response. Noise injected directly into the latent z (bypassing the encoder) isolates predictor and cost contributions. Metrics: the slope of the planner's cost surface under latent perturbations and the latent robust radius, defined as the largest perturbation that keeps the cost ranking stable.

Layer 5 — Task resolution. The control-relevant information retained in the latent. Metrics: the L2/cosine transition-resolution ratio R_d in Equation (3) and the inverse-dynamics linear-probe R^2, a controllability proxy in the spirit of linear-probe analysis [1].

Reading guidance. Layers 1, 2, 4, and 5 supply descriptive evidence used throughout this paper (the per-task radar in Figure 4 and the full base profile in Table 10). Layer 3 is the only layer whose two full-coverage metrics — the fragility ratio and the multi-step rollout drift — are used as cross-checkpoint predictors in the partial-correlation analysis of Section 4.5; the other layers are kept descriptive because either their probe definition is not single-valued per checkpoint (Layers 1, 4) or their metric is saturated in some tasks (Layers 2, 5).

3.4 Cross-checkpoint validation protocol

To rule out spurious correlations we adopt two complementary analyses on the same LeWM PushT sweep:

LeWM $n = 9$ sweep with partial correlation. All 9 LeWM checkpoints per task (base + `std_max` $\in \{0.01, \ldots, 0.08\}$). Compute Spearman ρ and *partial* Spearman conditioned on `std_max`. The partialling step matters: many diagnostics correlate with control performance only because both quantities co-vary with the training noise level along the sweep. The relevant question is whether a diagnostic retains a residual checkpoint-quality signal after removing the monotone sweep trend associated with `std_max`.

We report both the raw Spearman and the partial-on-`std_max` quantities. The raw ρ tells you "which checkpoint over the entire sweep behaves better"; the partial ρ tells you whether a diagnostic retains a residual checkpoint-quality signal after removing the monotone trend associated with `std_max`. The two questions are distinct, and we will see in Section 4.5 that they have markedly different answers for the same metric. Each reported partial Spearman value is paired with a 95% percentile bootstrap confidence interval ($B = 1000$ with-replacement resamples of the checkpoint rows within scope; seed = 42). The bootstrap output is released as `assets/paper1_data/partial_corr_bootstrap_20260523.json`, and the regeneration script is `tools/build_partial_corr_bootstrap.py`. PLDM provides a first second-family replication in Section F; broader cross-architecture variants such as frozen/pretrained visual features remain follow-up work.

4 Experiments

4.1 Setup

Tasks. PushT (2D pushing), TwoRoom (2D navigation), Reacher (2D arm), Cube (3D manipulation).

Baselines. LeWM-base (no noise) and LeWM+noise (8-level sweep).

Checkpoints. The 36 checkpoints in this paper correspond to one trained model per (task, `std_max`) configuration.

Evaluation seeds. Each checkpoint is evaluated with 3 evaluation seeds (42 / 43 / 44), with 100 trajectories per seed.

Evaluation protocol. All success rates in this paper — unperturbed and noised, across all 36 checkpoints (4 tasks × {base, `std_max` 0.01–0.08}) — are computed under a single protocol: $n = 3$ seeds (42/43/44), `num_eval = 100` trajectories per seed (300 trajectories per condition per checkpoint). We use *clean* only as the conventional artifact/condition name for unperturbed original evaluation images. Every cell of Tables 1, 2, 3 is mean ± across-seed population std over $n = 3$ and is sourced from the released aggregate, `assets/paper1_data/canonical_evals_20260517.json`; raw per-seed files live alongside each checkpoint as `<ckpt>/eval_results/<cond>_seed{42,43,44}_metrics.txt`. Success-rate tables report across-seed population std, while correlation intervals use checkpoint-level bootstrap CIs; these quantify different uncertainty sources. The diagnostic source-of-truth for Figure 5, Table 5, Table 6, and Table 7 is `assets/paper1_data/canonical_diagnostics_20260517.json`.

Hardware. Single NVIDIA H800 (80 GB).

Figures. Six main figures (Figures 1 to 6) are produced by `tools/paper1_figs.py`.

4.2 JEPA control fragility under visual corruptions

Table 1 reports LeWM-base success rates under unperturbed and noised evaluation (mean ± std across 3 seeds × 100 evaluations).

Table 1: LeWM-base under test-time visual corruptions (mean ± std; 3 seeds × 100 evaluations).

Task	unperturbed	px+goal 0.05	px+goal 0.08	unperturbed → 0.08 drop
TwoRoom	94.00 ± 3.56	61.33 ± 5.31	50.00 ± 1.41	**−44.00**
PushT	86.33 ± 2.36	12.00 ± 4.55	4.67 ± 2.05	**−81.67**
Reacher	58.67 ± 1.25	27.00 ± 5.10	15.00 ± 2.16	**−43.67**
Cube	66.67 ± 2.62	53.33 ± 3.30	46.33 ± 3.68	**−20.33**

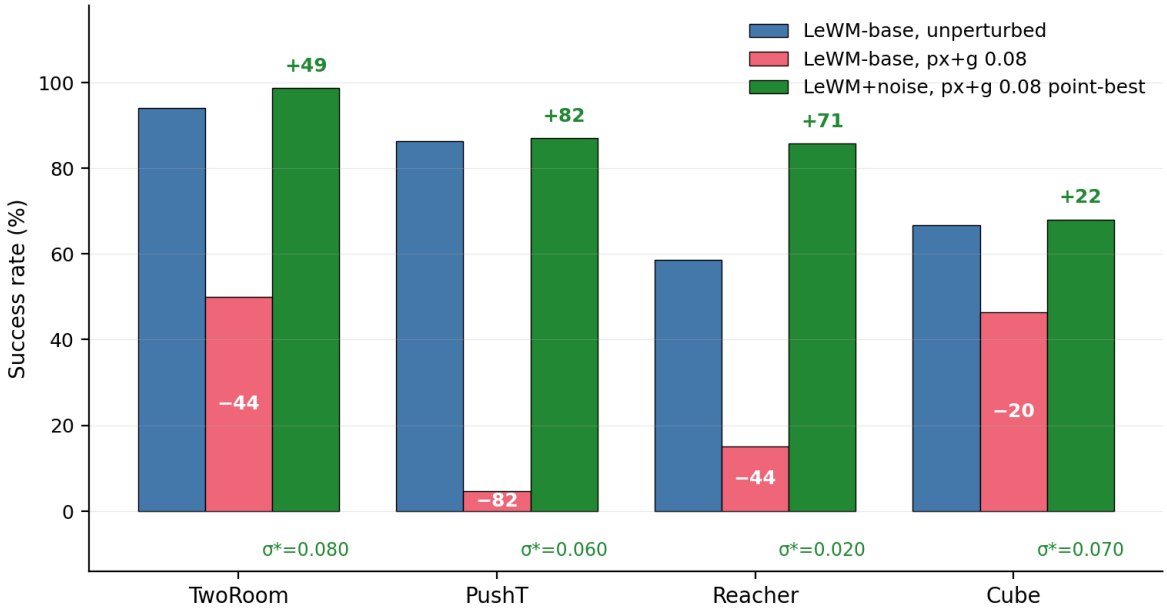


Figure 1: Visual-corruption cliff in LeWM and recovery by noise training (success rate, 4 tasks). Blue: LeWM-base on unperturbed images. Red: LeWM-base under px+g 0.08 noise. Green: the px+g 0.08 point-best LeWM+noise configuration. Annotations show base drop and recovery in points; σ^* marks each task's px+g 0.08 point-best `std_max`.

LeWM-base is strong on unperturbed images (especially TwoRoom and PushT), but visual std = 0.05 applied to pixels and goal jointly already produces large drops on all tasks: PushT loses 74+ pts (down to near-random 4.67% at std = 0.08), TwoRoom 30+ pts, Reacher 30+ pts, Cube $\sim$ 13 pts (at std = 0.05). This is a large closed-loop effect rather than a marginal representation-space shift. The drop pattern across tasks is informative: Cube degrades least (−20.33 pt at std = 0.08) — structured manipulation is less sensitive in this protocol — while PushT degrades most catastrophically (−81.67 pt), confirming that contact-heavy continuous control is most sensitive to visual precision.

The same direction of failure is visible on PLDM but with task-dependent strength. PLDM trained without noise augmentation drops sharply on PushT (75.33% ± 3.68 unperturbed to 10.00% ± 2.16 at px+goal 0.08, −65.33 pt) and TwoRoom (96.67% ± 1.70 to 51.67% ± 2.05, −45.00 pt), while Reacher and Cube have much smaller no-noise-training gaps (−3.33 and −4.33 pt; Table 14). The noise-training signatures still carry over: TwoRoom recovers into a high-noise plateau, PushT has a large `std_max` ≈ 0.03 recovery, and Cube responds weakly (Section F).

4.3 Noise augmentation closes the gap — at the cost of task-specific tuning

Tables 2 and 3 report the complete 8-level sweep across tasks. All cells are success rate ± across-seed std (in pts), aggregated under the unified $n = 3$ seeds (42/43/44) × 100 trajectories protocol — 300 trajectories per cell.

Table 2: LeWM+noise sweep, unperturbed (**clean** condition-name) success rate (%).

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	94.00 ± 3.56	86.33 ± 2.36	58.67 ± 1.25	66.67 ± 2.62
0.01	93.67 ± 3.30	88.00 ± 3.74	61.67 ± 2.49	69.33 ± 0.47
0.02	95.00 ± 2.83	88.33 ± 2.62	85.67 ± 2.49	60.00 ± 1.63
0.03	96.33 ± 3.30	$89.67 \pm 1.70^{\dagger}$	78.67 ± 1.25	65.00 ± 1.63
0.04	96.33 ± 2.05	89.33 ± 2.05	84.00 ± 2.94	69.00 ± 3.74
0.05	96.00 ± 2.83	80.67 ± 4.78	70.00 ± 2.16	59.33 ± 0.94
0.06	96.67 ± 2.05	89.33 ± 2.05	$86.00 \pm 2.94^{\dagger}$	66.67 ± 2.05
0.07	96.00 ± 1.63	85.67 ± 3.09	83.67 ± 3.30	67.67 ± 0.94
0.08	$98.33 \pm 0.47^{\dagger}$	88.33 ± 2.87	84.00 ± 0.82	62.33 ± 1.25

Table 3: LeWM+noise sweep, **px+goal 0.08** success rate (%).

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	50.00 ± 1.41	4.67 ± 2.05	15.00 ± 2.16	46.33 ± 3.68
0.01	87.67 ± 1.89	43.33 ± 3.09	46.00 ± 1.63	51.33 ± 5.79
0.02	93.33 ± 0.94	71.33 ± 3.68	$85.67 \pm 1.70^{\ddagger}$	60.67 ± 0.47
0.03	94.67 ± 2.87	83.00 ± 3.74	73.67 ± 0.47	67.33 ± 1.89
0.04	95.00 ± 2.45	81.33 ± 2.87	80.00 ± 1.41	67.00 ± 3.56
0.05	95.67 ± 2.36	75.00 ± 6.48	68.00 ± 3.56	59.67 ± 2.05
0.06	96.67 ± 2.49	$87.00 \pm 3.74^{\ddagger}$	84.67 ± 4.03	65.00 ± 2.94
0.07	96.33 ± 2.05	82.33 ± 4.64	81.33 ± 1.25	$68.00 \pm 1.41^{\ddagger}$
0.08	$98.67 \pm 0.94^{\ddagger}$	85.33 ± 2.62	83.00 ± 4.32	60.33 ± 0.94

Notation. † marks the unperturbed point-best in that task column; ‡ marks the px+goal 0.08 point-best. Table 4 and Figure 4 use a separate term, *representative diagnostic checkpoint*, for the checkpoint on which the full diagnostic suite was executed. Because many neighbouring settings differ by ≤ 3 pts and overlap in across-seed std, we treat the optima as plateaus unless the gap is clearly larger than the seed-level variability.

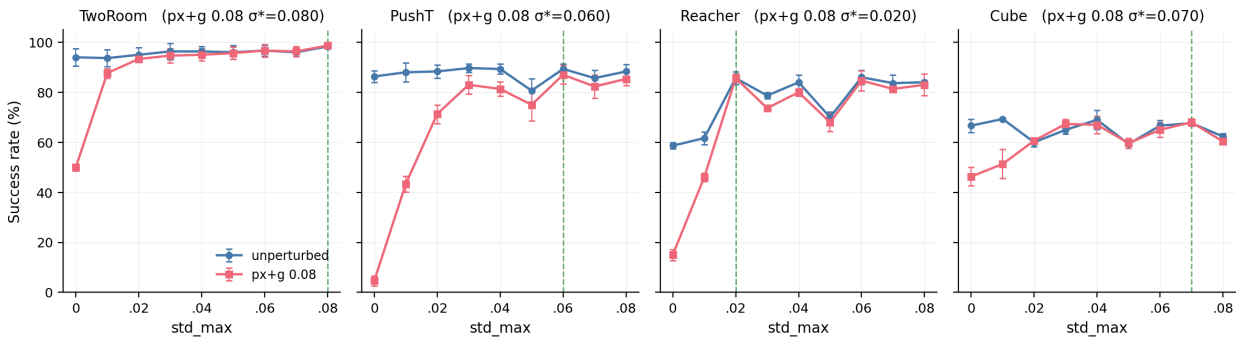


Figure 2: Noise-training sweep: unperturbed vs corrupted evaluation per task. Dashed vertical lines mark each task's px+g 0.08 point-best σ^*. No single `std_max` is jointly optimal across tasks.

Three observations follow.

(1) No single `std_max` is jointly optimal; unperturbed and robust optima can dissociate within a task. TwoRoom peaks globally at `std_max` = 0.08 (98.33 / 98.67); unperturbed success rises monotonically with noise. PushT peaks on unperturbed evaluation at `std_max` = 0.03 but on robustness at `std_max` = 0.06 (87.00 vs. 0.02's 71.33; +15.67 pt). Reacher lies on a 0.02–0.06 plateau: the unperturbed point-best is at `std_max` = 0.06 (86.00), while px+g 0.08 point-best is at `std_max` = 0.02 (85.67). At `std_max` = 0.01 unperturbed success is 61.67 vs base 58.67 — within across-seed std ($\sim$ 2.5 pts), so the data support *"low noise is statistically equivalent to base"*, not "low noise hurts". Cube responds least: unperturbed success is non-monotonic, and px+g 0.08 improves only in the 0.03–0.07 range.

(2) Per-task tuning is necessary, not optional. Optimal `std_max` varies substantially: TwoRoom unperturbed/corrupted point-best 0.08, PushT unperturbed point-best 0.03 / px+g 0.08 point-best 0.06, Reacher unperturbed point-best 0.06 / px+g 0.08 point-best 0.02, Cube px+g 0.08 point-best 0.07 with a shallow unperturbed plateau around 0.01 / 0.04 / 0.07. Global input-side noise is the strongest "global" form of invariance pressure, but closing the corruption gap requires per-task tuning cost.

(3) Tasks form a clear sensitivity ordering at the extremes. PushT is clearly most sensitive (−81.67 base drop), Cube least sensitive (−20.33), while TwoRoom (−44.00) and Reacher (−43.67) are effectively tied around a 44-pt drop. Recovery does not scale with sensitivity: TwoRoom recovers most fully (+48.67 pt at `std_max` = 0.08), Cube least (+21.67 pt). Input-side global noise is most effective on visually redundant tasks and gives diminishing returns on structured manipulation.

Plotting the same sweep as an (unperturbed, corrupted) trajectory per task (Figure 3) makes the trade-off visually explicit: each task's sweep curve moves from base far below the $y = x$ diagonal toward the upper-right, with per-task curvature determined by how much corruption-robustness gain a task can buy from a given drop in unperturbed performance. Figure 3 is the *behavioural projection* of the invariance–resolution trade-off; Table 4 in Section 4.4 is its *representation-level projection*, and Table 9 in Section 5.2 consolidates both into a per-task summary.

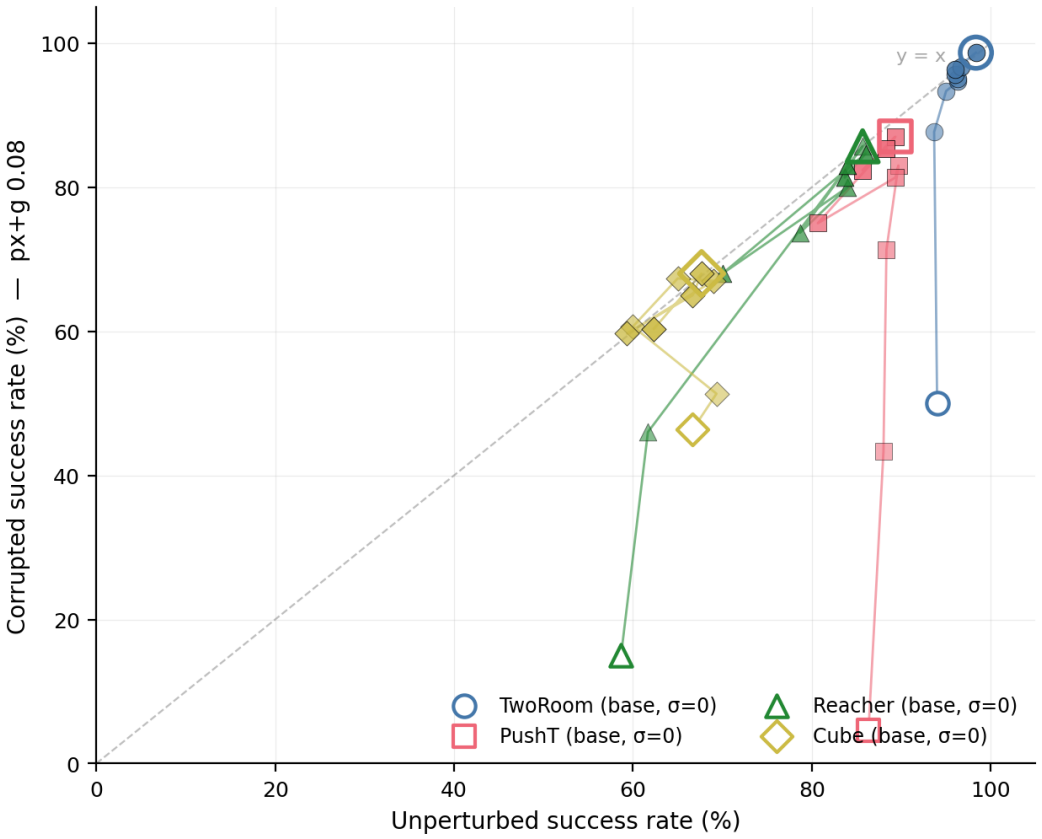


Figure 3: Per-task Pareto trajectory of (unperturbed, corrupted) evaluation under noise sweep. Open marker = base; solid markers = `std_max` sweep (colour-by-`std_max`); ringed marker = px+g 0.08 point-best. Dashed line $y = x$.

4.4 Diagnostic analysis: why global noise is not a silver bullet

Where the per-task sweep curves of Section 4.3 are the behavioural face of the invariance–resolution trade-off, the diagnostics in this subsection are its representational face: which latent properties move under noise training, and whether the movement is bounded by each task's resolution requirements. Table 4 compares core diagnostic metrics on LeWM-base versus the per-task representative noise-trained diagnostic checkpoint.

Table 4: Representation diagnostics: LeWM-base vs. a representative noise-trained diagnostic checkpoint per task. The σ pick per task (0.08 / 0.02 / 0.06 / 0.01) is the ckpt on which the diagnostic suite was originally executed. Under the unified 3-seed $\times$ 100 eval protocol the PushT unperturbed point-best shifts to `std_max` $= 0.03$ (within ± 2 pt of `std_max` $= 0.02$); we keep the diagnostics on the `std_max` $= 0.02$ checkpoint because the compression-vs.-resolution pattern this table illustrates is robust within this neighbourhood.

Metric	TwoRoom base	TwoRoom noise (0.08)	PushT base	PushT noise (0.02)	Reacher base	Reacher noise (0.06)	Cube base	Cube noise (0.01)
`clean_nn_cos_dist_median`	0.0449	0.0281	0.2360	0.1051	0.0633	0.0676	0.1856	0.1879
`clean_effective_rank`	47.60	33.59	76.42	42.85	61.04	65.92	73.25	71.83
`transition_resolution_ratio_l2`	0.7216	0.6055	0.3015	0.2800	0.3704	0.3791	0.4847	0.4629
`transition_resolution_ratio_cos`	0.5538	0.3780	0.0868	0.0800	0.1351	0.1399	0.2347	0.2168
`id_probe_r2`	0.2889	−0.0573	0.7739	0.7500	0.1621	0.1729	0.6657	0.6720
`action_mean_pred_shift_norm`	0.5329	0.4482	0.1283	0.1200	0.2518	0.2585	0.2364	0.2320
`predictor_rollout_T8_l2`	18.62	17.90	18.65	16.50	15.17	0.44	20.20	19.25

Notes. (i) Reacher and Cube representative diagnostics are taken from the corresponding checkpoint's per-tool JSON outputs under the diagnostics directory (max-std = 0.1, history-only noise); (ii) the Reacher representative rollout drift of 0.44 (a 35$\times$ reduction from base 15.17) is not a recording error: Reacher's representative checkpoint trains under `std_max` $= 0.06$, which is enough to collapse the multi-step predictor drift while keeping single-step task-resolution metrics flat — precisely the dissociation that motivates the analysis in Section 4.5. Cube's representative drift ($19.25 \approx$ base 20.20) is the opposite extreme.

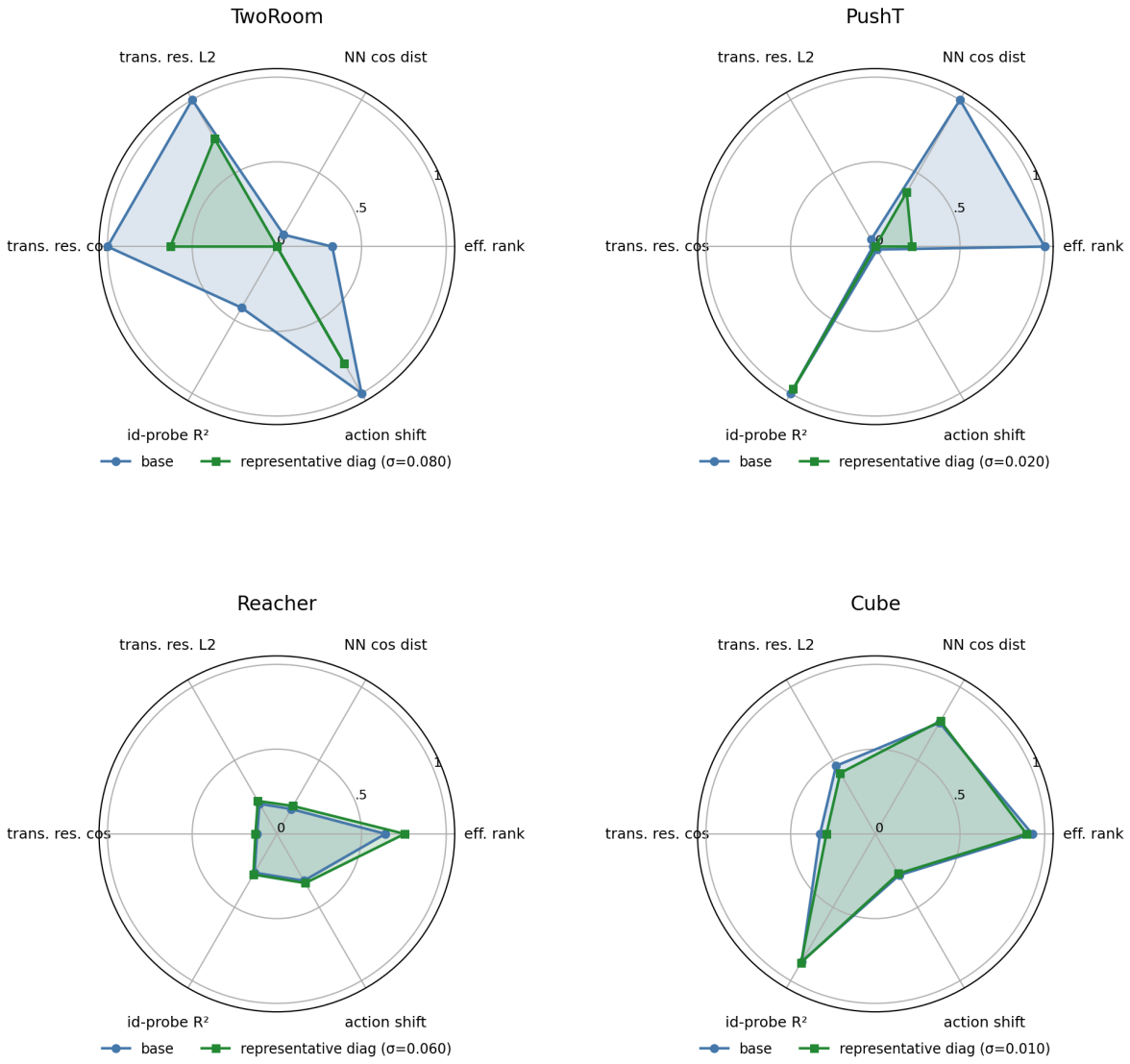


Figure 4: Per-task diagnostic radar: base vs representative noise-trained diagnostic checkpoint on 6 metrics, min-max normalised across all 4 tasks.

Mechanistic reading.

- **TwoRoom.** Low-dimensional, discrete, visually redundant. Compressing the representation (effective rank 47.6 → 33.6) is acceptable and even beneficial: a more compact latent space is easier to plan in.
- **PushT.** Continuous contact requires fine-grained pose resolution. Even at the representative diagnostic checkpoint (`std_max` = 0.02), the L2 transition-resolution metric already trends slightly downward. At heavier noise (e.g. 0.06) it drops further, erasing contact-transition keyframes.
- **Reacher.** Low-dimensional continuous reaching does not show a resolution collapse in Table 4: effective rank, transition resolution, and the controllability probe remain flat or slightly improve. The notable change is the large reduction in multi-step predictor drift (15.17 → 0.44), matching the later finding that Reacher's residual corruption-drop signal lives in the rollout metric rather than in the single-step fragility ratio.
- **Cube.** Cube is moderately resolution-demanding but less fragile than PushT. The representative

checkpoint leaves rank, transition resolution, controllability probe, and rollout drift almost unchanged, consistent with Cube's smaller visual-corruption cliff and the moderate residual signal of multi-step drift in Table 6.

- **Predictor-rollout caveat.** A drop in multi-step predictor rollout drift is not unambiguously good news. It can also mean the latent has become more *predictable* without being more *controllable*: predictor stability bought by sacrificing resolution.

4.5 Cross-checkpoint correlation analysis

Table 5 reports task-specific cross-checkpoint correlations on the LeWM $n = 9$ sweep using the unified 3-seed $\times$ 100 evaluation.

We analyse two single-value-per-checkpoint diagnostic metrics that have full coverage on all 9 checkpoints per task: the fragility ratio, defined in Section 3.3 as the single-step predictor target shift normalised by the nearest-neighbour cosine distance at the diagnostic's max-std injection, and the corresponding multi-step rollout L2 drift. Both are released in `assets/paper1_data/canonical_diagnostics_20260517.json`, derived from each checkpoint's predictor-sensitivity diagnostic JSON, and are pure checkpoint-level quantities (independent of the evaluation protocol).

Table 5: LeWM $n = 9$ sweep: per-task Pearson r / Spearman ρ vs corruption drop (unperturbed − px+g 0.08). Eval values come from the unified 3-seed $\times$ 100 protocol.

Metric	TwoRoom (r/ρ)	PushT (r/ρ)	Reacher (r/ρ)	Cube (r/ρ)
`predictor_target_to_nn_cos_ratio_at_max_std`	+0.96/+0.72	**−0.54**/**−0.77**	+0.97/+0.35	+0.36/+0.21
`predictor_rollout_T8_l2_at_max_std`	+0.89/+1.00	**+0.99**/**+0.88**	**+0.99**/**+0.87**	+0.92/+0.54

Reading. Unconditional correlations are large because both diagnostic metrics and the corruption drop are driven by the same upstream variable — `std_max`. The relevant test is partial correlation conditioned on `std_max` (Table 6).

Table 6: Partial Spearman ρ(metric, corruption drop | `std_max`), $n = 9$ per task.

Metric	TwoRoom	PushT	Reacher	Cube
`predictor_target_to_nn_cos_ratio_at_max_std`	— (rank ties)	+0.06	−0.12	+0.14
`predictor_rollout_T8_l2_at_max_std`	— (rank ties)	−0.03	**+0.79**	+0.50

The TwoRoom partial estimates are undefined because saturating success rates produce rank ties that make residualisation singular at $n = 9$. PushT, Reacher and Cube give clear readings: after removing the monotone sweep trend associated with `std_max`, the **fragility metric carries essentially no information about the size of the corruption drop** on any task; the **multi-step predictor drift** still carries signal on Reacher (+0.79) and weak signal on Cube (+0.50). The Reacher partial ρ is the only non-trivial residual correlation in the entire matrix.

Table 7: Spearman ρ (unconditional) and partial Spearman ρ conditioned on `std_max`, for the fragility ratio on the PushT $n = 9$ LeWM sweep under the unified 3-seed $\times$ 100 eval. CIs are the percentile 95% interval over $B = 1000$ bootstrap resamples (with replacement, seed = 42; see Section 3.4). The `std_max`-only top four rows are univariate sweep diagnostics with no metric/outcome pairing, so no CI is reported.

Quantity	Spearman ρ	95% bootstrap CI
ρ(`std_max`, metric)	+0.83	—
ρ(`std_max`, unperturbed)	0.00	—
ρ(`std_max`, px+goal 0.08)	+0.82	—
ρ(`std_max`, corruption drop)	−0.93	—
ρ(metric, unperturbed) unconditional	−0.33	[−0.95, +0.62]
ρ(metric, unperturbed) | `std_max` partial	**−0.59**	[−0.97, −0.10]
ρ(metric, px+g 0.08) unconditional	+0.55	[−0.28, +0.86]
ρ(metric, px+g 0.08) | `std_max` partial	**−0.41**	[−0.77, +0.00]
ρ(metric, corruption drop) unconditional	−0.77	[−0.95, −0.12]
ρ(metric, corruption drop) | `std_max` partial	+0.06	[−0.00, +0.25]

PushT $n = 9$ detailed view (fragility ratio). Interpretation:

1. **After removing the monotone trend associated with `std_max`, the metric still carries a meaningful residual checkpoint-quality signal.** Partial ρ on unperturbed evaluation is −0.59 and on px+goal 0.08 is −0.41 — both negative, both meaningful at this sample size. On the $n = 9$ PushT sweep, lower fragility ratio aligns with better unperturbed *and* better noisy success beyond what the sweep-level `std_max` trend already explains.
2. **The metric does NOT predict the unperturbed-to-corrupted gap.** The unconditional $\rho(\text{metric}, \text{drop}) = -0.77$ looks impressive, but partialling out `std_max` collapses it to +0.06 (sign-flipped, near zero; 95% bootstrap CI [−0.00, +0.25], including 0). The drop's strong correlation with the metric is a mediated effect: `std_max` drives both the metric ($\rho = +0.83$) and the drop ($\rho = -0.93$). After the `std_max` trend is removed, the metric tells you nothing new about how much the *gap* between unperturbed and noisy performance will be.
3. **Note the unconditional-vs-partial sign reversal on unperturbed evaluation.** ρ(metric, unperturbed) is only −0.33 unconditionally — unperturbed success rate is roughly flat across the PushT sweep under the 3-seed protocol (range 80.67–89.67), so `std_max` barely moves unperturbed performance. The partial correlation *strengthens* to −0.59 once the sweep-level `std_max` trend is removed, exactly because the residual signal is what the metric tracks.
4. **Practical reading.** The toolkit is useful for mechanism localisation and checkpoint selection after controlling for the sweep-level `std_max` effect. It is *not* a substitute for actually training and evaluating under corruptions when the robustness gap is the quantity of interest.

4.5.1 Unperturbed control fidelity and corruption robustness as separable signals

The partial-correlation analysis above settles the interpretation:

- The metric is a **residual checkpoint-quality signal beyond the sweep-level `std_max` effect** — a lower fragility ratio means better control on *both* unperturbed and noisy evaluation (partial $\rho = -0.59$ / -0.41 on PushT).
- The metric **does not isolate noise robustness** — its apparent correlation with the gap between unperturbed and noisy success is fully mediated by `std_max` (partial $\rho = +0.06$).

The PushT scatter (Figure 5) makes both halves visible. Panel (a) plots metric vs unperturbed success (unconditional Spearman $\rho = -0.33$; the residual trend after conditioning on `std_max` is what the partial correlation captures). Panel (b) plots metric vs corruption drop (unconditional

$\rho = -0.77$), but the colour-bar (encoding `std_max`) reveals the structure: low-`std_max` checkpoints (light blue) live at high drop, high-`std_max` checkpoints (dark blue) live at low drop, and the metric tracks `std_max` itself. After the `std_max` trend is removed, the metric does not separate small-drop from large-drop checkpoints.

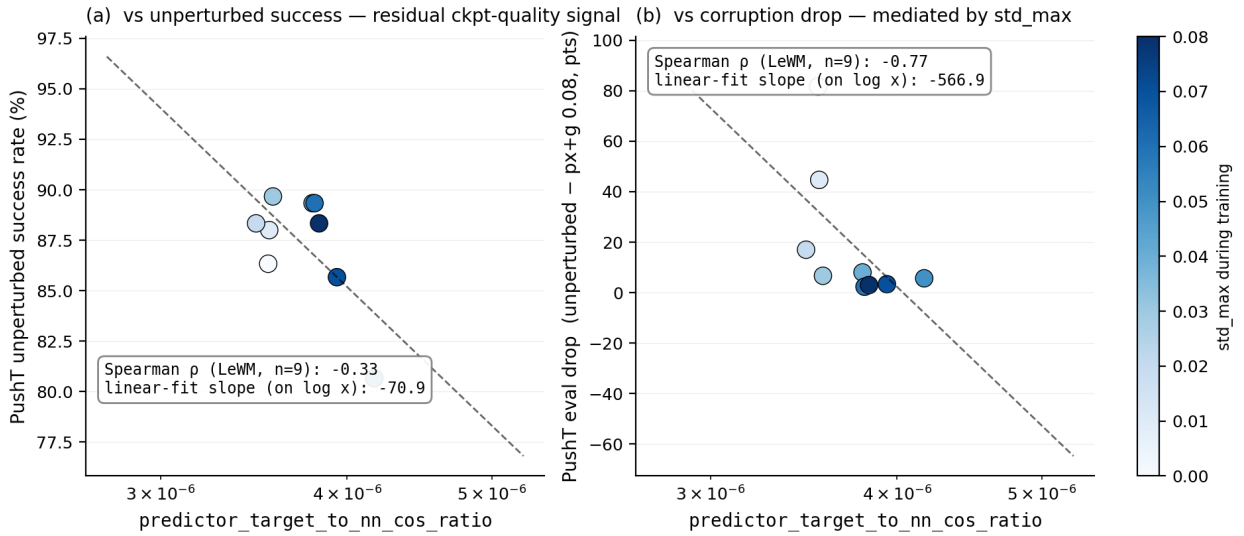


Figure 5: PushT $n = 9$ LeWM sweep: the fragility ratio is a checkpoint-quality predictor (a); the apparent corruption-drop correlation in (b) is mediated by `std_max` (encoded by the colour bar).

4.6 Mechanism attribution: where in the pipeline does noise cause failure?

Section 4.4 reports *what* is compressed and Section 4.5 reports *which diagnostics* correlate with eval across checkpoints, but neither answers **"is the failure in the encoder, the predictor, or the cost surface?"** We address this with two complementary experiments.

4.6.1 Auxiliary cost-swap sanity check: cost alone is unlikely to explain the collapse

We ran a one-off eval-only cost swap on a single TwoRoom checkpoint outside the 36-checkpoint canonical evaluation table; full details are reported in Section E. Switching the CEM cost from cosine/normalized to mse/raw changes px+goal 0.03 success only from 36.0 to 42.0, still far below a separate unperturbed reference of 69.7 (separate $n = 300$ eval, single seed; see Section E). As a sanity check, this suggests that the cost function alone is unlikely to explain the visual-corruption collapse.

4.6.2 Latent-noise probing: encoder is the principal bottleneck

Directly injecting noise into the latent z (skipping the encoder) decouples encoder contributions from predictor + cost. Comparing diagnostic signals across two injection points (pixels vs. latent z, both history-only noise at max std):

- **PushT.** Multi-step input-space rollout drift has unconditional $\rho = +0.88$ vs corruption drop (Table 5), driven primarily by `std_max`; after removing the monotone `std_max` trend, the partial ρ collapses to -0.03. The single-step fragility ratio (unconditional $\rho = -0.77$; partial $+0.06$ after removing the same trend) tells the same story. Both encoder–predictor signals are mediated by

training noise; once that sweep-level effect is accounted for, neither isolates the corruption drop on PushT.

- **Reacher.** Multi-step rollout drift has unconditional $\rho = +0.87$ *and* partial $\rho = +\mathbf{0.79}$ vs corruption drop (95% bootstrap CI $[+0.00, +1.00]$, borderline-significant at the 95% level) — the only non-trivial residual correlation in the matrix. On Reacher, multi-step predictor drift carries genuine corruption-drop information beyond `std_max`.
- **Cube.** Multi-step rollout drift has unconditional $\rho = +0.54$ and partial $\rho = +0.50$ (95% CI $[-0.30, +1.00]$, marginal) — a moderate residual signal. Encoder–predictor drift partially explains Cube's small but non-zero corruption sensitivity.
- **TwoRoom.** Partial correlations are undefined because unperturbed success / drop are saturated across the sweep (rank ties). Unconditional ρ still indicates strong encoder–predictor sensitivity to `std_max`.

Table 8 summarises the three-layer attribution.

Table 8: Three-layer attribution by task (LeWM $n = 9$ sweep, unified 3-seed $\times$ 100 eval).

Task	Primary cause	Residual after removing the `std_max` trend
TwoRoom	encoder dominant (rank-saturated)	n/a
PushT	encoder + single-step predictor (both mediated by `std_max`)	no residual metric isolates corruption drop
Reacher	encoder + multi-step rollout	multi-step drift residual signal (partial $\rho = +0.79$)
Cube	encoder	moderate residual on multi-step drift (partial $\rho = +0.50$)

The common primary cause across the four tasks is **encoder shift transduced by the predictor**. The auxiliary cost-swap sanity check in Section 4.6.1 suggests that the cost function alone is unlikely to explain the collapse, but we do not treat that single-checkpoint ablation as a task-wide quantitative attribution. This is also the root reason Layer-5 task-resolution metrics carry strong signal in Section 4.4: once the encoder's latent-neighbourhood structure is corrupted beyond the NN distance scale, downstream predictor and planner are already operating on the wrong neighbourhood.

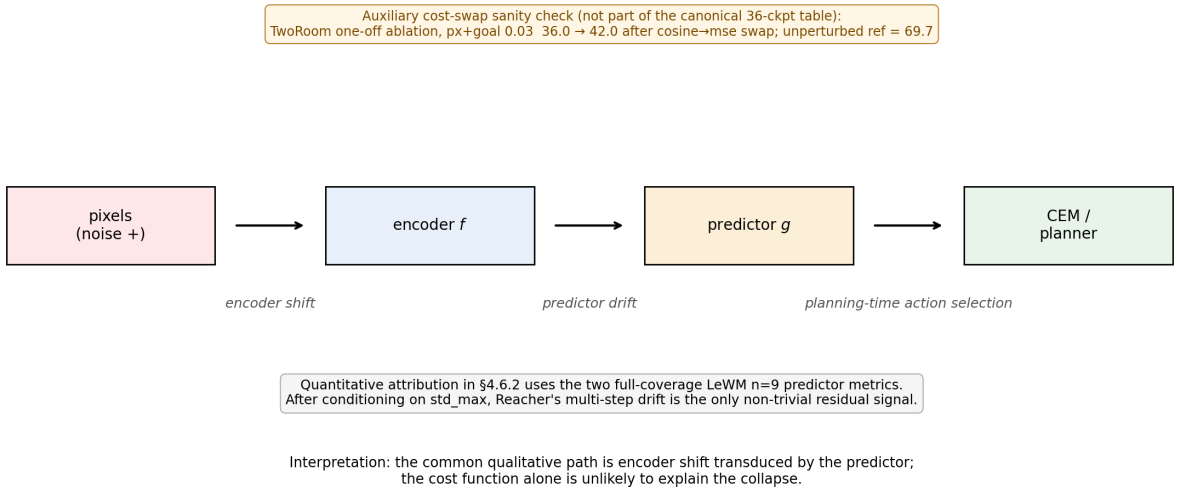


Figure 6: Mechanism schematic. Pixels perturb the encoder, the predictor transduces that shift into planning-relevant latent drift, and CEM acts on the resulting latent geometry. Quantitative attribution in §4.6.2 uses the two full-coverage LeWM $n = 9$ predictor metrics; after conditioning on `std_max`, Reacher's multi-step drift is the only non-trivial residual signal.

5 Discussion

5.1 Rethinking the "JEPA invariance" narrative

The data challenge an implicit community assumption: latent prediction alone is not sufficient to confer invariance to visual noise. The invariance a JEPA encoder acquires is *in-distribution invariance* — invariance to visual patterns present in the training distribution. When test-time inputs carry high-frequency pixel noise not seen during training, the topology of the latent space breaks down, the predictor outputs incorrect future states, and the planner fails.

This should be read alongside VJEPA [15], which reports that JEPA-family models keep signal-recovery $R^2 > 0.84$ under a synthetic Noisy-TV distractor. That positive result is compatible with ours because the evaluation modalities differ: signal-recovery probes only need the latent to preserve recoverable state information, whereas closed-loop control success demands a representation and predictor that support precise latent-space planning and action optimisation.

5.2 The invariance–resolution trade-off — manifestation and scope

The trade-off has the following four-task signature in this paper:

- **TwoRoom.** Background wall colour / texture is pure visual redundancy; discarding it does not impair navigation.
- **PushT.** The pixel changes during T-block / arm contact carry critical force and pose information; discarding them blinds the planner.
- **Reacher.** Low-dimensional continuous control; visual encoding of joint angle requires moderate resolution.
- **Cube.** Object pose and grasp-point spatial relations need moderate resolution, but Cube itself is largely insensitive to global noise (action sequence is structured; visual–action coupling is predictable).

Task-specificity explains why we observe no single optimal noise level on the sweep grid. Table 9 consolidates the behavioural (Section 4.3) and representational (Section 4.4) evidence into a single per-task read of the trade-off.

Table 9: Where each LeWM task sits on the invariance–resolution trade-off. The two "demand" columns are qualitative task-property reads, not independent measurements; they index the underlying behavioural and representational evidence in Sections 4.3 and 4.4 and the operational definition in Section 3.3. Sweep signatures come from **??** and table 3; diagnostic readings from Table 4.

Task	Nuisance-invariance demand	Control-resolution demand	Observed sweep signature	Diagnostic reading at representative ckpt
TwoRoom	high (visual redundancy)	low–moderate (discrete navigation)	heavy-noise plateau ($\sigma^* = 0.08$)	compact latent acceptable; rank 47.6 → 33.6 without controllability loss
PushT	moderate	high (contact precision)	dissociated optima ($\sigma^*_{\text{unp}} = 0.03$, $\sigma^*_{\text{corr}} = 0.06$)	compression risks resolution; rank 76.4 → 42.9 with transition-resolution and inverse-dynamics-probe downward movement
Reacher	moderate	moderate	broad plateau ($\sigma^* \in 0.02$–0.06)	rollout drift carries the residual signal more than rank or single-step resolution
Cube	low–moderate	moderate	weak response ($\sigma^* = 0.07$)	structured action coupling buffers noise; small representation movement

Scope. Whether the trade-off generalises to other latent world-model architectures is an open question we do not fully address. Reconstruction-based world models such as DreamerV3 [12] explicitly model observations, while decoder-free latent MPC systems such as TD-MPC2 [13] may exhibit different compression dynamics. ViGMO [22] observes related task-specific noise sensitivity on DeepMind Control (DMC). Exponential-moving-average (EMA) target JEPA variants (I-JEPA / V-JEPA lineage) and variational JEPA may exhibit different dynamics. Our mechanism claim is LeWM-centred, with PLDM used as a second-family replication for the task-level fragility/recovery signature and the PushT partial-correlation null; universalising the compression chain exceeds the present evidence.

5.3 When the diagnostic toolkit works — and when it does not

Three scope conditions should be stated up-front so practitioners know when to use the toolkit and when to fall back to direct evaluation.

Scope 1: the toolkit ranks residual checkpoint quality, not corruption-specific robustness. Section 4.5.1 shows that on PushT, after conditioning on `std_max`, the strongest cross-checkpoint diagnostic (the fragility ratio) has partial Spearman $\rho = -0.59$ with unperturbed success and $\rho = -0.41$ with px+goal 0.08 success. It does *not* isolate corruption-specific robustness: the partial correlation with the unperturbed-to-corrupted drop is only $+0.06$. Use the toolkit to pick the strongest checkpoint after removing the sweep-level `std_max` trend; do not use it as a substitute for an actual corruption eval if your downstream question is robustness.

Scope 2: tasks with weak per-state controllability variance fall outside the toolkit's strict regime. Reacher and TwoRoom do not produce a diagnostic-vs-eval Spearman ρ that survives the partial-correlation criterion. Both tasks lack enough residual spread after accounting for `std_max` to distinguish "good" from "bad" checkpoints via a label-free metric. The toolkit can describe what these models have compressed (Table 4) but cannot predict relative checkpoint quality.

Scope 3: cross-checkpoint diagnostics cannot recover what training did not provide. The largest determinant of the corruption drop is whether the model saw noise during training — not which fragility ratio it happens to score on. This is the structural reason $\rho(\text{metric}, \text{drop})$ is weak even when $\rho(\text{metric}, \text{unperturbed})$ is strong: noise vs no-noise training puts each checkpoint on a completely different (unperturbed, corrupted) curve (cf. Figure 3), and no static cross-checkpoint diagnostic can substitute for that training-time choice.

The toolkit therefore has a precise scope: it is an *unperturbed-evaluation auxiliary* that lets you select among checkpoints on tasks with strong per-state controllability variance (PushT, Cube), assuming the training protocol is already fixed. It is useful for mechanism localisation and checkpoint selection within a fixed protocol, but it is not a robustness oracle.

5.4 Practical guidance: how to choose `std_max` on a new task

Our sweep suggests a simple operational recipe:

1. Inspect the no-noise baseline's fragility ratio as a screening diagnostic, not as a robustness oracle. Very small values (PushT / Cube regime) indicate that local encoder–predictor shift is small relative to the unperturbed nearest-neighbour scale, but they do not by themselves rank corruption sensitivity.
2. Check effective rank, transition-resolution ratio, and task semantics together. PushT combines high rank and high controllability ($\texttt{id\_probe\_r}^2 = 0.7739$), so noise should be swept with unperturbed performance as a guardrail. TwoRoom is visually redundant and retains high L2 transition separability ($R_{\mathrm{L2}} = 0.7216$), so a wider sweep up to 0.08+ is reasonable.
3. Use *two endpoints* in evaluation (unperturbed and max-noise). Checking only one misses one of the optima: PushT's unperturbed optimum at 0.03 vs robustness optimum at 0.06 is the clearest example.
4. Under compute budget, start with a coarse 4-level sweep ($\{0.01, 0.03, 0.05, 0.07\}$), then refine around the best unperturbed/corrupted candidates. The coarse grid is a screening step, not a guarantee of locating the exact point-best `std_max`.

5.5 Limitations and future directions

Limitation 1 — Two backbone families validated; broader JEPA variants remain open. The toolkit and the trade-off concept are introduced on LeWM; the PLDM sweep in Section F replicates the task-level signatures and the partial-correlation null on PushT. Section F also reports the PLDM full-diagnostic check, which exposes an important mechanism boundary: PLDM recovery checkpoints largely preserve rank, transition-resolution, and inverse-dynamics-probe statistics while reducing multi-step predictor drift. We therefore keep the main compression-chain interpretation LeWM-centred rather than claiming it as a universal mechanism. EMA-target JEPA variants (I-JEPA / V-JEPA lineage) and variational JEPA may exhibit different noise responses; we did not run them.

Limitation 2 — Gaussian pixel noise is the controlled training axis. The main sweep uses Gaussian pixel noise because it gives a scalar `std_max` axis for unperturbed/corrupted trade-off analysis. Section G adds a no-noise-checkpoint blur stress test and shows a corruption-specific profile: pixels+goal blur primarily collapses TwoRoom on both LeWM and PLDM (-63.67 pt and -68.33 pt), while PushT, Cube, and PLDM-Reacher are comparatively stable and LeWM-Reacher degrades mainly at the strongest blur endpoint. Visual fragility therefore generalises beyond the Gaussian-noise axis, but the task ordering and recovery profile are corruption-specific; blur-specific training and recovery remain follow-up work.

Limitation 3 — Diagnostic framework is empirical, not theoretical. Reacher and TwoRoom fail the partial-correlation criterion for *all* metrics, exposing where the empirical framework breaks down. We do not have a closed-form theory that predicts which tasks support a cross-checkpoint diagnostic and which do not; the current framework is a label-free *screen* with the empirical scope

conditions in Section 5.3. Whether the residual checkpoint-quality signal we observe on PushT extrapolates to longer-horizon or out-of-grid `std_max` regions remains an open question.

Additional ablation — automated transition reweighting is not a substitute for noise training. As a sanity check we also tested a scale-preserving heteroscedastic negative-log-likelihood (NLL) formulation in which a learned per-transition σ-head down-weights high-error transitions. The result is informative as a negative finding: TwoRoom unperturbed success reaches 99.67% but high-noise robustness lags noise training; PushT unperturbed success *collapses* from 86.33% to 13.33% because contact-control transitions have high prediction error and are exactly the transitions σ-weighting would discard. Full data are reported in Section D. This is the sharpest expression of the resolution side of the invariance–resolution trade-off: a global compression pressure that does not distinguish nuisance variation from action-relevant variation collapses the latter precisely where it is most needed ("hard $\neq$ unimportant" in contact control). The methodological direction — per-token controllers that preserve resolution rather than discard it — is left to follow-up work.

Future direction 1 — Per-token adaptive consistency. Sections 4.5 and 4.6 identify the strongest signal — the fragility ratio — as a checkpoint-level scalar. Whether its per-token variant can serve as a per-token consistency controller is an open methodological question, under investigation in follow-up work.

Future direction 2 — Broader cross-architecture replication. PLDM gives one second-family replication. Frozen/pretrained visual-feature models and EMA-target JEPA variants remain the next useful external checks.

Future direction 3 — Theoretical grounding. Reformulating the trade-off in an information-bottleneck / rate-distortion language is an attractive direction; we did not pursue it because the empirical phenomenology may not yet be stable enough for formal modelling.

6 Conclusion

This paper provides a systematic diagnostic study of visual-corruption robustness in JEPA + CEM world models, using LeWorldModel as the main microscope and PLDM as a second-family replication. Three findings:

1. Severe visual-corruption fragility in JEPA + CEM control. Without noise-aware training, PushT exhibits a near-collapse under Gaussian pixel noise and TwoRoom shows severe degradation; latent prediction alone does not confer visual-corruption robustness in this protocol.
2. Global noise augmentation has a boundary. It effectively closes the corruption gap, but no single `std_max` is jointly optimal on the sweep grid; task differences are large, and within a single task unperturbed and robustness optima can dissociate.
3. A five-layer diagnostic protocol reveals the underlying mechanism. On LeWM, noise-augmentation gains arise from representation compression, but excessive compression destroys the resolution required for control — the *invariance–resolution trade-off*. The PLDM cross-method check in Section F reproduces the same task-level recovery signature with a different diagnostic profile, most consistently accompanied by reduced multi-step predictor drift while rank and resolution are largely preserved. The compression chain is therefore a LeWM-family mechanism rather than a universal claim. When used cross-checkpoint, the strongest single diagnostic (the fragility ratio)

predicts unperturbed control quality rather than corruption-specific robustness; we delineate this scope explicitly.

This paper does not propose a new training algorithm. Its contribution is a systematic body of empirical evidence, a reusable diagnostic toolkit, and an explicit delineation of what cross-checkpoint diagnostics can and cannot predict.

Acknowledgements

The complete code and data are available at `https://github.com/qun-team/wm_exp`. We thank the LeWM authors for releasing the baseline framework on which this study builds.

References

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. ICLR 2017 Workshop Track / arXiv preprint, 2017.

[2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video (v-jepa). *Transactions on Machine Learning Research / arXiv:2404.08471*, 2024.

[5] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[6] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.

[7] Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 702–703, 2020.

[8] T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[9] Carlos E. García, David M. Prett, and Manfred Morari. Model predictive control: Theory and practice — a survey. *Automatica*, 25(3):335–348, 1989.

[10] Quentin Garrido, Randall Balestriero, Laurent Najman, and Yann LeCun. Rankme: Assessing the downstream performance of pretrained self-supervised representations by their rank. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

[11] Hafez Ghaemi, Eilif Benjamin Muller, and Shahab Bakhtiari. seq-jepa: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[12] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025.

[13] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[14] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[15] Yongchao Huang. Vjepa: Variational joint embedding predictive architectures as probabilistic world models. *arXiv preprint arXiv:2601.14354*, 2026.

[16] Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[17] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

[18] Dirk P. Kroese, Sergey Porotsky, and Reuven Y. Rubinstein. The cross-entropy method for continuous multi-extremal optimization. *Methodology and Computing in Applied Probability*, 8(3):383–407, 2006.

[19] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[20] Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026.

[21] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[22] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[23] Yuping Qiu, Rui Zhu, and Ying-cong Chen. Improving joint embedding predictive architecture with diffusion noise (n-jepa). *arXiv preprint arXiv:2507.15216*, 2025.

[24] Ashwath Radhachandran, Vedrana Ivezić, Shreeram Athreya, Ronit Anilkumar, Corey W. Arnold, and William Speier. Us-jepa: A joint embedding predictive architecture for medical ultrasound. *arXiv preprint arXiv:2602.19322*, 2026.

[25] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *Proceedings of the European Signal Processing Conference (EUSIPCO)*, 2007.

[26] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022.

[27] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: A case for planning with latent dynamics models. In *WRL@ICLR 2025*, 2025.

[28] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 20827–20840, 2022.

[29] Alex Tamkin, Margalit Glasgow, Xiluo He, and Noah Goodman. Feature dropout: Revisiting the role of augmentations in contrastive learning. In *Proceedings of NeurIPS*, 2023.

[30] Jayden Teoh, Manan Tomar, Kwangjun Ahn, Edward S. Hu, Pratyusha Sharma, Riashat Islam, Alex Lamb, and John Langford. Next-latent prediction transformers learn compact world models. *arXiv preprint arXiv:2511.05963*, 2025.

[31] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.

[32] Tongzhou Wang and Phillip Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.

[33] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[34] Denis Yarats, Ilya Kostrikov, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[35] Junbo Zhang and Kaisheng Ma. Rethinking the augmentation module in contrastive learning: Learning hierarchical augmentation invariance with expanded views. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16650–16659, 2022.

A Experimental details

A.1 Environments

- **PushT.** 2D continuous pushing; 20,000 expert episodes; $\sim$196 steps/episode; action dim 2 (direction + magnitude); 224×224 RGB.
- **TwoRoom.** 2D continuous navigation; 10,000 episodes; $\sim$92 steps; action dim 2; 224×224 RGB.
- **Reacher.** 2D arm control (DeepMind Control Suite); 10,000 episodes; 200 steps; action dim 2.
- **Cube.** 3D cube manipulation (OGBench); 10,000 episodes; 200 steps; action dim 7.

A.2 Noise augmentation implementation

The actual implementation is `utils.py::AddNormalizedGaussianNoise`. Key points: (i) noise is added on ImageNet-normalized tensors and must be divided by the per-channel std to retain "pixel-space std" semantics; (ii) sampling is per-frame independent over the leading frame dims, not per-batch; (iii) all sweeps in this paper fix `noise_prob` = 1.0 and `std_min` = 0 and vary only `std_max`.

```python
class AddNormalizedGaussianNoise:
    """Per-frame independent. Each frame draws Bernoulli(noise_prob) then
    std ~ Uniform(std_min, std_max). Pixel-space std is converted to
    normalized space by dividing by per-channel std before adding."""
    def __init__(self, std_min, std_max, noise_prob=1.0):
        self.std_low, self.std_high, self.noise_prob = std_min, std_max, noise_prob
        self.channel_std = torch.as_tensor(IMAGENET_STD) # (C,)

    def __call__(self, x):
        if self.std_high <= 0 or self.noise_prob <= 0:
            return x
        leading = x.shape[:-3]
        stds = torch.empty(leading, device=x.device, dtype=x.dtype).uniform_(
            self.std_low, self.std_high)
        if self.noise_prob < 1.0:
            mask = (torch.rand(leading, device=x.device) < self.noise_prob).to(x.dtype)
            stds = stds * mask
        per_frame_scale = stds.view(*leading, 1, 1, 1)
        channel_factor = (1.0 / self.channel_std.to(x.device, x.dtype)).view(
            *([1] * len(leading)), -1, 1, 1)
        scale = per_frame_scale * channel_factor # pixel-space std -> normalized
        return x + torch.randn_like(x) * scale
```

A.3 Evaluation protocol

Unperturbed evaluation uses no noise, 100 trajectories per seed × 3 seeds (42/43/44), totalling 300 trajectories per condition. Noisy evaluation adds Gaussian noise of std $\in \{0.05, 0.08\}$ to both pixels and the goal image under the same 3-seed protocol. Success criterion is task-specific (PushT: T-block pose match; TwoRoom: target region; Reacher: joint-angle match; Cube: cube position match).

A.4 Compute

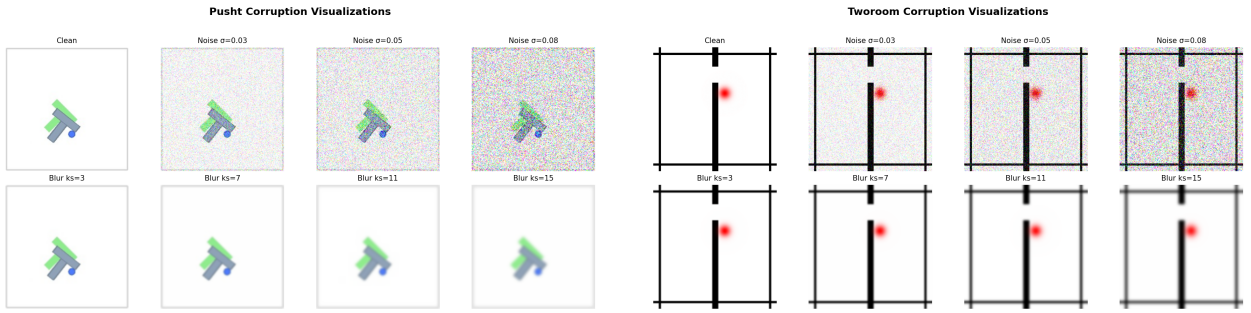
Training runs on a single NVIDIA H800 (80 GB) and follows the LeWM training schedule released with the baseline.

A.5 Main-figure rendering

The six main figures are produced by `tools/paper1_figs.py`; run `python-mtools.paper1_figs--out-dirassets/paper1_figs`. The script reads `assets/paper1_data/canonical_evals_20260517.json` together with `assets/paper1_data/canonical_diagnostics_20260517.json`; no checkpoint or model loading is required.

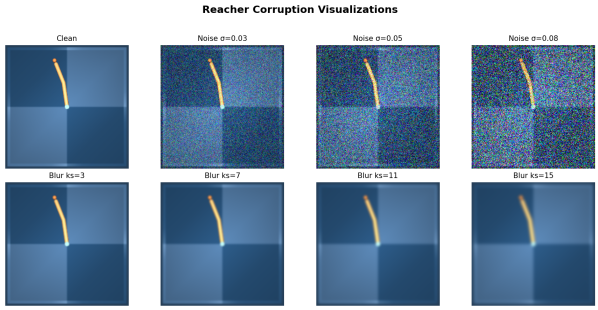
A.6 Corruption visualizations

Figure 7 shows representative renderings of the two evaluated corruption families, applied to a sample observation from each of the four tasks. The Gaussian-noise severities ($\sigma \in \{0.03, 0.05, 0.08\}$) match the px+goal evaluation grid used in Sections 4.2 and 4.3 (main sweep); the Gaussian-blur kernel sizes ($k \in \{3, 7, 11, 15\}$) match the stress test in Section G. Each panel is a single 224×224 RGB frame from the corresponding task's evaluation rollout, after corruption and before encoder normalisation.

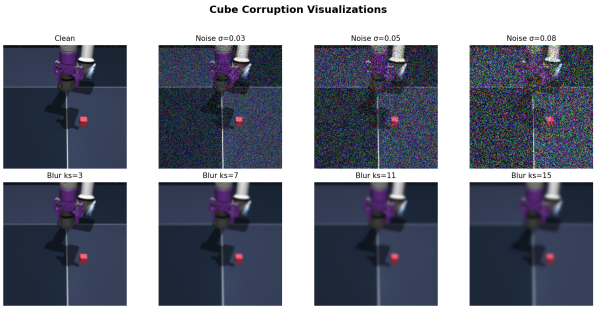


(a) PushT

(b) TwoRoom



(c) Reacher



(d) Cube

Figure 7: Corruption visualizations on the four tasks. Each task panel is a 2×4 grid: **Row 1** additive Gaussian pixel noise (the *Clean* reference and $\sigma \in \{0.03, 0.05, 0.08\}$); **Row 2** Gaussian blur at kernel sizes $k \in \{3, 7, 11, 15\}$ (sigma derived via the OpenCV / torchvision auto rule). PushT at $\sigma = 0.08$ is visually close to pure noise, consistent with the near-collapse control success rate of 4.67% reported in **??**.

B Full diagnostic profile of LeWM-base on four tasks

Table 10 summarises every core diagnostic on the four LeWM-base checkpoints. Data are taken from each checkpoint's per-tool JSON outputs under `eval_results/diagnostics/` (the `geometry`, `task`, `predictor` and `latent_noise` families).

Table 10: Full LeWM-base diagnostics across four tasks.

Layer / Metric	TwoRoom	PushT	Reacher	Cube	Unit / note
Encoder geometry					
`clean_nn_cos_dist_median`	0.0449	0.2360	0.0633	0.1856	cosine distance
`clean_pair_cos_dist_median`	0.9904	1.0228	1.0252	1.0193	pair-wise cos dist
`clean_effective_rank`	47.60	76.42	61.04	73.25	effective rank
Noise sensitivity					
`noise_angle_deg_median` (@std= 0.05)	5.51°	1.33°	3.22°	1.40°	median angular shift
`noise_to_nn_cos_ratio_median`	0.1031	0.011	0.0249	0.016	noise/NN cos ratio
`robust_radius_std`	0.0142	0.0537	0.0142	0.0356	critical noise std
`first_risk_std`	>0.08	>0.08	>0.08	>0.08	first high-risk std
`noise_angle_slope_deg_per_std`	1085.8	284.8	831.7	327.0	°/std angular gain
`geometry_flag`	balanced	robust	balanced	robust	geometry label
Task resolution					
`transition_resolution_ratio_l2`	0.7216	0.3015	0.3704	0.4847	L2 resolution ratio
`transition_resolution_ratio_cos`	0.5538	0.0868	0.1351	0.2347	cos resolution ratio
`id_probe_r2`	0.2889	0.7739	0.1621	0.6657	linear probe R^2
`id_probe_r2_min`	0.2599	0.6786	0.1366	0.0972	min probe R^2
`lidar_rank`	46.06	13.95	45.90	42.46	LiDAR rank proxy
Action effect					
`action_mean_pred_shift_norm`	0.5329	0.1283	0.2518	0.2364	action-perturb mean shift
`action_perturb_pred_shift_corr`	0.2847	0.2873	0.4042	0.2559	shift × $\|a\|$ corr
Predictor stability					
`predictor_rollout_T8_l2`	18.62	18.65	15.17	20.20	T=8 rollout L2 drift
`predictor_target_to_nn_cos_ratio_at_max_std`	1.51e-4	3.54e-6	2.67e-5	3.39e-6	max std target/NN ratio
Latent noise					
`cka_linear_at_max_std`	0.1986	0.5536	0.3085	0.1814	CKA unperturbed vs noisy
`latent_cost_surface_slope_z`	635.31	1.3886	599.45	0.6208	goal latent cost slope

C Heteroscedastic-loss formulation

The scale-preserving heteroscedastic NLL referenced in Section 5.5 (the "Additional ablation" paragraph) and detailed in Section D is:

$$\mathcal{L}_{\text{hetero}} = \tfrac{1}{2}\,\exp(-s_t)\,\|z_{t+1} - \hat{z}_{t+1}\|^2 + \tfrac{1}{2}\,s_t\,, \tag{4}$$

where s_t is the σ-head-predicted log-variance. During training $\exp(-s_t)$ acts as an automatic weight: high-error transitions are down-weighted and low-error transitions are up-weighted. When $s_t \equiv 0$ the loss reduces to plain MSE.

D Heteroscedastic-loss negative result (data)

This appendix records the full data behind the "Additional ablation" paragraph in Section 5.5. The σ-head is trained jointly with the predictor; gradient is detached on the σ path so the mean prediction path is exactly LeWM MSE when σ is constant.

Table 11: Heteroscedastic-loss evaluation.

Task / model	Unperturbed	goal 0.05	pixels 0.05	px+goal 0.05	goal 0.08	px+goal 0.08
TwoRoom LeWM-base	94.00	73.33	72.33	61.33	58.67	50.00
TwoRoom LeWM+noise point-best	98.33	98.00	98.33	98.00	98.67	98.67
TwoRoom hetero	**99.67**	85.33	96.67	84.67	73.33	55.33
PushT LeWM-base	86.33	38.00	17.00	12.00	11.33	4.67
PushT LeWM+noise point-best	**89.33**	87.67	88.00	88.33	89.67	87.00
PushT hetero	**13.33**	7.67	7.67	7.67	9.67	6.00

TwoRoom hetero reaches 99.67% on unperturbed evaluation — consistent with the prior that low-dimensional discrete tasks benefit from stronger invariance / clustering — but lags noise training on high-noise robustness. **PushT hetero unperturbed success is** 13.33% — **a method-level failure**, not a robustness trade-off.

Table 12: Representation diagnostics of the hetero-loss failure.

Metric	TwoRoom base	TwoRoom hetero	PushT base	PushT hetero
`clean_nn_cos_dist_median`	0.0449	0.0281	0.2360	0.1051
`clean_effective_rank`	47.60	33.59	76.42	42.85
`transition_resolution_ratio_cos`	0.5538	0.3780	0.0868	0.0101
`transition_resolution_ratio_l2`	0.7216	0.6055	0.3015	**0.1023**
`id_probe_r2`	0.2889	−0.0573	0.7739	**0.2678**
`action_mean_pred_shift_norm`	0.5329	0.4482	0.1283	0.0841
`predictor_rollout_T8_l2`	18.62	17.90	18.65	14.01

Hetero-loss compresses the representation in both tasks. For TwoRoom (low-dimensional, discrete, redundant), compression is acceptable. For PushT, the L2 transition-resolution ratio collapses from 0.3015 to 0.1023 and `id_probe_r2` from 0.7739 to 0.2678 — task-relevant state information is erased. The drop in multi-step rollout drift is not good news either: the latent has become more *predictable* without being more *controllable*.

Mechanism. The σ-head correctly learns per-transition difficulty (high σ on contact frames in PushT, on doorway crossings in TwoRoom, etc.), but using σ as an automatic weight to down-weight high-error transitions misclassifies the contact-and-control-critical states of PushT as "unimportant" and erases them. This is a direct demonstration of the broader trade-off lesson: in contact-heavy control, **hard** $\neq$ **unimportant**.

This finding motivates a follow-up direction we are currently exploring: per-token *consistency* (not loss-reweighting) controllers driven by detached difficulty signals, which preserve the mean prediction path's gradient distribution. That work is independent of this paper's diagnostic study and will be reported separately.

E One-off cost-swap sanity check

This appendix records the eval-only cost-swap ablation referenced in Section 4.6.1. It is **not** part of the 36-checkpoint canonical evaluation table and uses a separate unperturbed reference (num_eval = 300), so we report it only as a sanity check rather than a pooled statistic.

Table 13: One-off eval-only cost swap on a TwoRoom checkpoint, std = 0.03 pixels+goal. Reference row reports a separate unperturbed eval of the same checkpoint (num_eval = 300).

Variant	Cost type	Cost space	Success rate
A (default)	cosine	normalized	36.0
B (swap)	mse	raw	42.0
Unperturbed reference (same ckpt)	—	—	69.7

Swapping cost recovers only +6 pt (36 → 42), still far below the unperturbed reference (69.7). The narrow reading is therefore: **a sanity check suggests the cost function alone is unlikely to explain the visual-corruption collapse.**

F Cross-method replication: PLDM noise sweep on four tasks

This appendix records the second method-family on which the noise sweep and the cross-checkpoint correlation analysis replicate. PLDM follows the joint-embedding predictive line from Sobal et al. [26]; the concrete latent-dynamics planning baseline is from Sobal et al. [27]. We use the implementation distributed through the `stable-worldmodel` baseline suite [20]. Training and evaluation follow the LeWM canonical protocol bit-for-bit: per-frame Gaussian pixel noise via `utils.py::AddNormalizedGaussianNoise`, `std_max` $\in \{0.01, \ldots, 0.08\}$, three evaluation seeds (42/43/44), 100 trajectories per seed.

Coverage. The PLDM release is complete for the same grid as the LeWM canonical sweep: 4 tasks × 9 configurations (no-noise baseline plus `std_max` $\in \{0.01, \ldots, 0.08\}$), three evaluation seeds (42/43/44), and 100 trajectories per seed. The PLDM eval, predictor-diagnostic, full-diagnostic, and cross-method-correlation JSON files are listed in the data manifest; we do *not* merge them into the LeWM canonical files used by Tables 1–7.

No-noise-trained PLDM cliff. PLDM reproduces the no-noise-trained visual-corruption cliff on PushT and TwoRoom, while Reacher and Cube show much smaller drops under this PLDM training recipe (Table 14). This supports a method-family reading of the failure mode without claiming every task has the same cliff magnitude.

Table 14: PLDM baseline trained without input-noise augmentation on all four tasks. Success rate mean ± population std, 3 seeds × 100 trajectories.

Task	unperturbed	px+goal 0.05	px+goal 0.08	unperturbed → 0.08 drop
TwoRoom	96.67 ± 1.70	68.33 ± 3.30	51.67 ± 2.05	−45.00
PushT	75.33 ± 3.68	43.67 ± 4.64	10.00 ± 2.16	−65.33
Reacher	82.67 ± 1.70	82.33 ± 0.94	79.33 ± 0.94	−3.33
Cube	52.67 ± 3.30	51.33 ± 2.49	48.33 ± 2.87	−4.33

PLDM full sweep — unperturbed evaluation. Table 15 reports the per-task unperturbed success rate at every trained `std_max` level. Table 16 reports the corresponding px+goal 0.08 robustness.

Table 15: PLDM+noise sweep, unperturbed (**clean** condition-name) success rate (%). † marks the per-task unperturbed point-best.

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	96.67 ± 1.70	75.33 ± 3.68	82.67 ± 1.70	52.67 ± 3.30
0.01	96.33 ± 0.94	72.33 ± 4.19	78.00 ± 0.82	51.33 ± 1.25
0.02	97.33 ± 0.47	71.33 ± 3.86	82.33 ± 2.36	53.33 ± 1.70
0.03	96.67 ± 1.70	$76.67 \pm 7.54^{\dagger}$	83.00 ± 3.74	52.67 ± 3.30
0.04	97.67 ± 0.47	68.67 ± 5.25	$85.00 \pm 2.16^{\dagger}$	54.00 ± 2.94
0.05	97.67 ± 1.25	74.33 ± 3.40	72.00 ± 1.41	54.00 ± 2.16
0.06	98.00 ± 0.82	70.33 ± 6.18	79.00 ± 0.82	$56.00 \pm 4.97^{\dagger}$
0.07	98.00 ± 0.82	66.67 ± 8.38	80.67 ± 2.62	53.67 ± 3.68
0.08	$98.33 \pm 0.94^{\dagger}$	68.00 ± 1.41	80.33 ± 1.25	54.00 ± 1.63

Table 16: PLDM+noise sweep, **px+goal 0.08** success rate (%). ‡ marks the per-task px+goal 0.08 point-best.

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	51.67 ± 2.05	10.00 ± 2.16	79.33 ± 0.94	48.33 ± 2.87
0.01	89.33 ± 3.30	16.67 ± 4.03	78.00 ± 1.63	53.00 ± 1.63
0.02	93.33 ± 0.94	40.00 ± 0.82	79.33 ± 2.05	52.33 ± 3.40
0.03	94.33 ± 1.70	$72.00 \pm 2.94^{\ddagger}$	79.33 ± 2.36	55.67 ± 4.11
0.04	98.00 ± 0.00	62.33 ± 7.41	$81.33 \pm 1.70^{\ddagger}$	53.00 ± 3.56
0.05	$98.33 \pm 0.94^{\ddagger}$	69.33 ± 4.92	59.67 ± 3.30	53.00 ± 1.63
0.06	97.33 ± 1.25	69.67 ± 5.25	76.00 ± 1.41	53.33 ± 3.09
0.07	96.67 ± 0.47	67.00 ± 9.09	77.67 ± 3.30	52.00 ± 0.00
0.08	97.00 ± 0.82	66.33 ± 4.78	80.67 ± 2.36	$56.33 \pm 1.25^{\ddagger}$

Reading. Four quantitative observations are useful for interpreting PLDM as a second method family:

1. **TwoRoom (visually redundant) benefits from heavy noise on both methods.** PLDM px+goal 0.08 success rises from 51.67% at the no-noise baseline to 98.33% at `std_max` = 0.05, then remains on a high-noise plateau. This mirrors the LeWM reading that TwoRoom tolerates strong invariance.
2. **PushT (contact-heavy) exhibits the corruption cliff and large noise-training recovery.** PLDM without noise augmentation drops by 65.33 pt under px+goal 0.08; training with `std_max` = 0.03 recovers px+goal 0.08 success to 72.00% (+62.00 pt over the no-noise baseline). The

unperturbed and px+goal 0.08 point-bests are co-located at `std_max` = 0.03 on PLDM, unlike LeWM where the robust point-best moves to a heavier-noise checkpoint.

3. **Reacher is already robust under no-noise-trained PLDM.** The no-noise PLDM drop is only 3.33 pt, and the point-best for both unperturbed and px+goal 0.08 sits at `std_max` = 0.04 (85.00% unperturbed, 81.33% px+goal 0.08). We therefore treat Reacher as a low-cliff external baseline rather than evidence for a universal noise-training gain.
4. **Cube (structured 3D manipulation) remains weakly noise-responsive.** PLDM Cube unperturbed success spans 51.33–56.00% and px+goal 0.08 spans 48.33–56.33% across the full grid. This is the same mildest manifestation of the trade-off seen on LeWM.

Method-specific details. PLDM's PushT unperturbed point-best (76.67% at `std_max` = 0.03) is lower than LeWM's (89.67% at `std_max` = 0.03), so the external baseline is not a performance leaderboard. It is a robustness check: the cliff-and-recovery shape is reproduced, while the exact unperturbed/robust optimum location is method-dependent. In particular, the LeWM PushT optimum dissociation should be read as a LeWM-family signature, not a cross-method law.

Cross-method partial correlations. We repeat the fragility ratio partial-correlation analysis for PLDM on all four tasks. The headline PushT null replicates in the limited sense that we do not see stable evidence for a non-zero residual corruption-gap association: PLDM has unconditional $\rho(\text{metric}, \text{corruption drop}) = -0.78$ but partial $\rho(\text{metric}, \text{corruption drop}) \mid \texttt{std\_max} = -0.14$ (95% bootstrap CI $[-1.00, +0.87]$), and the joint LeWM+PLDM PushT partial after conditioning on `std_max` and method is +0.11 (95% CI $[-0.54, +0.71]$). The intervals are wide and include zero. Other tasks show task-specific residuals, so we use the full table as a scope boundary rather than as a universal diagnostic claim.

Table 17: PLDM fragility ratio partial correlations, $n = 9$ per task. The first three partial columns condition on `std_max` within PLDM; the final column uses the joint LeWM+PLDM sample ($n = 18$) and conditions on both `std_max` and a method dummy. Numbers come from the released PLDM correlation JSON.

Task	PLDM n	unperturbed partial	px+goal 0.08 partial	corruption-drop partial	joint corruption-drop partial
TwoRoom	9	−0.26	−0.85	+0.87	+0.56
PushT	9	−0.22	+0.04	−0.14	+0.11
Reacher	9	−0.68	−0.86	+0.14	+0.34
Cube	9	−0.36	−0.05	−0.41	−0.13

PLDM full-diagnostic mechanism check. We also run the same five diagnostic layers on the PLDM sweep and release the resulting per-checkpoint summary in `canonical_full_diagnostics_pldm_20260523.json`. The compact base-vs-representative view in Table 18 shows a different mechanism profile from the LeWM compression chain in Table 4: PLDM's representative recovery checkpoints do *not* exhibit a broad collapse in rank, transition resolution, or inverse-dynamics probe. The most consistent movement is instead a reduction in multi-step predictor drift. This is useful precisely because it separates two claims: the task-level fragility/recovery signature replicates across method families, while the exact diagnostic mechanism is architecture-dependent.

Table 18: PLDM full-diagnostic check, no-noise baseline → representative noise-trained checkpoint. Representatives are each task's PLDM px+goal 0.08 point-best ckpt (TwoRoom `std_max` = 0.05, PushT `std_max` = 0.03, Reacher `std_max` = 0.04, Cube `std_max` = 0.08). Values come from the released full-diagnostics JSON.

Task	representative `std_max`	effective rank	transition ratio L2	inverse-dynamics probe R^2	rollout T8 L2
TwoRoom	0.05	74.78 → 72.80	0.8188 → 0.8270	0.3233 → 0.3273	20.36 → 1.19
PushT	0.03	130.13 → 131.90	0.4710 → 0.4778	0.7570 → 0.7479	20.25 → 2.82
Reacher	0.04	117.52 → 114.99	0.5053 → 0.5104	0.2150 → 0.1960	1.45 → 1.23
Cube	0.08	134.80 → 124.79	0.6395 → 0.6401	0.5428 → 0.5531	18.98 → 0.62

What this strengthens, and what it does not. PLDM strengthens C1 by showing that the four task signatures are not tied to one LeWM checkpoint family. It strengthens C4 only in the precise PushT sense stated in the main text: after removing the sweep-level `std_max` effect and the method offset, the local predictor-sensitivity diagnostic does not isolate corruption-specific robustness. The TwoRoom and Reacher PLDM residuals are deliberately left visible in Table 17. The TwoRoom corruption-drop partial = +0.87 does not contradict the headline null: both unperturbed and noisy TwoRoom success are saturated near 97–98% across the PLDM sweep, so the corruption-drop range is only a few points in absolute terms and the partial picks up rank-order noise inside that band rather than a controllable robustness signal. The Reacher px+goal 0.08 partial = −0.86 similarly reflects a tight noisy-eval cluster (59.67–81.33% across the sweep) and tracks noisy success rather than the corruption gap. Both residuals are task-specific artefacts of saturation and should not be converted into a robustness oracle.

Mechanism boundary. Table 18 is the reason we do not promote the LeWM compression chain to a universal theorem. On LeWM, noise-trained representatives can compress effective rank and reduce transition-key resolution; on PLDM, the same task-level robustness pattern is accompanied mainly by reduced rollout drift with largely preserved rank/resolution probes. The shared conclusion is therefore about *control-time visual fragility and noise-training recovery*; the internal diagnostic route is method-specific.

G Cross-corruption sanity check: no-noise-trained blur evaluation

This appendix addresses the narrow concern that the observed visual fragility is an artefact of Gaussian noise only. We evaluate the LeWM and PLDM baselines trained without input-noise augmentation under Gaussian blur with kernel sizes $3, 7, 11, 15$, applied to pixels, goal images, or both. This is an **eval-only** stress test: no blur-trained checkpoint is included, and the blur data are not mixed into the Gaussian-noise sweep. Source: `canonical_blur_baselines_20260523.json`.

Table 19: Pixels+goal blur stress test on baselines trained without input-noise augmentation. Success rate mean ± population std over 3 evaluation seeds × 100 trajectories. Kernel sizes 11 and 15 are severe low-pass endpoints on 224×224 inputs, so they should not be read as mild corruptions.

Method	Task	unperturbed	$k = 3$	$k = 7$	$k = 11$	$k = 15$	worst drop
LeWM	TwoRoom	94.00 ± 3.56	39.33 ± 2.05	32.67 ± 3.09	30.33 ± 2.87	30.67 ± 0.47	-63.67
LeWM	PushT	86.33 ± 2.36	81.67 ± 4.50	74.00 ± 1.41	65.33 ± 2.49	58.67 ± 6.65	-27.67
LeWM	Reacher	58.67 ± 1.25	57.00 ± 6.38	45.67 ± 2.05	36.00 ± 4.90	20.33 ± 2.49	-38.33
LeWM	Cube	66.67 ± 2.62	65.00 ± 1.63	62.33 ± 2.62	57.33 ± 2.87	54.00 ± 2.16	-12.67
PLDM	TwoRoom	96.67 ± 1.70	38.33 ± 0.47	30.33 ± 2.87	28.33 ± 1.25	28.33 ± 1.70	-68.33
PLDM	PushT	75.33 ± 3.68	74.00 ± 2.94	72.00 ± 3.56	67.67 ± 4.03	62.33 ± 2.05	-13.00
PLDM	Reacher	82.67 ± 1.70	86.00 ± 2.16	80.00 ± 3.56	72.00 ± 2.16	68.00 ± 4.08	-14.67
PLDM	Cube	52.67 ± 3.30	53.00 ± 5.35	53.00 ± 2.83	50.67 ± 3.68	52.33 ± 5.44	-2.00

Reading. Blur is a different failure mode from Gaussian pixel noise. The clear collapse is TwoRoom: it falls below 40% already at $k = 3$ on both LeWM and PLDM and then saturates around 28–33% for $k = 7$–15. This reverses the Gaussian-noise ordering, where PushT is the most fragile task. The other tasks are more stable under blur: PLDM PushT, PLDM Reacher, and both Cube baselines lose at most about 15 pt at the strongest endpoint, while LeWM PushT degrades gradually and LeWM Reacher drops sharply only at $k = 15$. We therefore use blur only as an eval-only cross-corruption sanity check: visual fragility is not unique to Gaussian noise, but the task ordering and recovery profile are corruption-specific.